DA06 Functions and checks that fail loudly

Week 2, Python you need | Session 6 of 24 | Python for Data Analysis

Today's question. Can we trust our revenue calculation on cases we already know the answer to?

Objective. By the end: you can write a function that returns a result, and use assert to check it on known cases.

> **WHERE THIS IS USED**
> Payment and tax software relies on functions that must give the right answer on cases where the answer is already known. A check that fails loudly protects every customer. What known case would you use to test a discount function?

Before this session: DA05 Lists, dictionaries and loops. If you cannot explain the last example, use its error card with the instructor first.

Words for today

Word	What it means	Think of it like this
function	A named set of instructions you can use again.	A recipe with a name.
parameter	The name for an input inside a function.	The blank in a form.
argument	The actual value you give a function when you call it.	The answer you write in the blank.
return	Send the result back to where the function was called.	The cashier handing you the change.
assert	Stop with an error if a statement is False. Say nothing if it is True.	A friend checking the answer without saying well done.
edge case	A value at the boundary where a rule might break, such as zero.	A customer who buys nothing.

Block A. A function and a check

A function starts with def. print only shows something on the screen. return sends a result back so other code can use it. A passing assert prints nothing at all. A failing one stops with an error.

Predict. Predict what the function returns for 3 units at 2000 shillings, and what a passing assert prints. Write your guess on paper first.

Your guess, written before you run it

```
def revenue(units, price):
    return units * price

print(revenue(3, 2000))
```

```
assert revenue(3, 2000) == 6000
assert revenue(0, 2000) == 0
```

Run it, then compare. The cell prints 6000. The two asserts print nothing, which means both pass. units and price are the parameters. 3 and 2000 are the arguments.

Investigate.

1. What is the difference between return and print?
2. What does it mean that the asserts print nothing?

Modify. Add a third assert that checks revenue(5, 1000) equals 5000.

Block B. Edge cases and a guard

A share is a part divided by a whole. Dividing by zero is an edge case. A function can guard against it with an if before it divides.

Predict. Predict the result of share(12, 20) and of share(5, 0). Write your guess on paper first.

Your guess, written before you run it

```
def share(part, whole):
    if whole == 0:
        return 0
    return part / whole

print(share(12, 20))
print(share(5, 0))
assert share(12, 20) == 0.6
assert share(5, 0) == 0
```

Run it, then compare. The cell prints 0.6 and then 0. When the whole is zero the function returns before it divides, so no error appears.

Investigate.

1. Why does the function check the whole before it divides?
2. Is returning 0 for share(5, 0) always the right answer?

Modify. Add an assert that share(1, 4) equals 0.25.

Make

Write price_after_discount(price, percent) that returns the price after a percentage discount. Check it with assert for 0 percent, 10 percent and 100 percent on a price of 2000. Save the notebook and run all cells from the top.

Save your work as DA06. Keep the worked example and put your own attempt below it. Write two sentences: what you changed and how you checked it.

Error card

You see	It means	Fix
AssertionError	The statement after assert was False, so the function gave a different answer from the one you expected.	Print the function result for that input and compare it with your hand calculation.
The function shows None	The function has no return line, so it gives nothing back.	Add return in front of the value you want back.

Exit ticket

A function returns a result, and an assert checks that result on cases I already know.

AI use follows AI_USE_POLICY.pdf. In weeks 1 to 6 no AI-generated solutions are used for exercises, labs or build tasks. Practise for 20 to 30 minutes between classes by repeating the Make task with one value changed. If you are stuck for 30 minutes, save the error and bring it to class. A file saved on your laptop has not been shared until you use the route in START_HERE.pdf.